Verificación y Validación de Software · Licenciatura en Desarrollo de Software

Trabajo Práctico Integrador

Lineamientos de entrega y evaluación

requisitos, especificaciones técnicas, criterios de evaluación y condiciones de aprobación

Material de cátedra de uso interno — Escuela de Ciencia y Tecnología (ECyT-UNSAM). Su reproducción fuera del ámbito académico de la cátedra requiere autorización previa del docente.

Versión de documento 1.0.0

Índice de contenidos

Introducción

1. Objetivos

2. Temática del proyecto

3. Requisitos del Proyecto

4. Especificaciones Técnicas

5. Entrega

6. Consignas

7. Evaluación

8. Fecha de revisiones

9. Fecha de entrega

10. Lineamientos

11. Condiciones de aprobación

Introducción

Los trabajos prácticos integradores tienen como finalidad generar las capacidades y habilidades necesarias para interpretar claramente los objetivos de un problema de calidad de software y resolverlo aplicando una adecuada estrategia de verificación y validación, seleccionando y justificando las técnicas, los niveles de prueba y las herramientas que se consideren pertinentes. Los estudiantes deberán aplicar en el trabajo práctico integrador los contenidos, las técnicas, las herramientas y las actividades prácticas desarrolladas en la asignatura hasta la fecha de entrega; los temas que se dicten con posterioridad a esa fecha no serán exigibles. El alumno deberá hacer revisiones parciales de avances establecidos por el docente durante la cursada. Los docentes harán el seguimiento del alumno en cada entrega y exposición del trabajo práctico integrador, el cual se calificará con una nota individual en la instancia evaluatoria del mismo, definida en una fecha posterior a la exposición de los trabajos prácticos integradores de la asignatura, establecida por los docentes a cargo de la materia.

1. Objetivos

1. Aplicar los conceptos, las técnicas y las herramientas de verificación y validación de software vistos durante el transcurso del periodo del ciclo educativo sobre una aplicación backend desarrollada con el lenguaje de programación Java y el framework Spring Boot.
2. Desarrollar habilidades en el análisis de requerimientos, el diseño de casos de prueba (caja negra, caja gris y caja blanca), la automatización de pruebas, la gestión de defectos y la medición de la calidad del producto.
3. Desarrollar habilidades y el pensamiento crítico para evaluar la calidad de un producto de software, justificar las decisiones de la estrategia de pruebas y analizar los resultados obtenidos a partir de evidencias y métricas.
4. Los alumnos formarán equipos de trabajo de 3 colaboradores, que actuarán como un equipo de QA distribuyendo los roles y las tareas de testing, deberán compartir código y artefactos de prueba, expresar ideas y resultados técnicos de manera clara y comprensible, aprender a gestionar el tiempo y cumplir con los plazos en proyectos de verificación y validación.

2. Temática del Proyecto

El Trabajo Práctico Integrador estará orientado a la planificación, el diseño, la ejecución y la automatización de la verificación y validación de una aplicación backend desarrollada con Java y Spring Boot. Cada equipo actuará como el equipo de QA responsable de evaluar la calidad del sistema, de detectar y reportar sus defectos y de informar el estado del producto a partir de evidencias.

Los sistemas que desarrollaran los equipos, serán definidos y asignados por los docentes, y deberán establecer sus funcionalidades, reglas de negocio, validaciones y flujos suficientes para aplicar las técnicas y los niveles de prueba vistos en la asignatura (por ejemplo, operaciones de alta, baja, modificación y listado, cálculos, condiciones de decisión y estados).

El proyecto deberá presentar un alcance acotado que permita su planificación, ejecución y demostración durante el período establecido para el Trabajo Práctico Integrador. El equipo deberá priorizar qué funcionalidades probar a partir de un análisis de riesgos y justificar esa decisión en su estrategia de pruebas.

3. Requisitos del Proyecto

Estrategia y plan de pruebas: El equipo deberá elaborar un plan de pruebas que defina objetivo, alcance (qué se prueba y qué no se prueba), enfoque y estrategia, niveles y tipos de prueba, técnicas de diseño, criterios de entrada y de salida, entornos de prueba (desarrollo, QA, staging y producción), roles y responsabilidades, estimación y cronograma, riesgos, control de cambios, ciclo de release e indicadores (KPIs) de QA. Cada decisión deberá estar justificada.

Diseño y ejecución de pruebas: A partir de la estrategia definida, el equipo deberá seleccionar y aplicar los contenidos de la asignatura que hayan sido dictados hasta la fecha de entrega, justificando la elección y la profundidad con que se aplica cada uno. Los contenidos que se dicten con posterioridad a la fecha de entrega no serán exigibles.

Fundamentos y reporte de errores : aplicación de los conceptos de verificación y validación, testing estático (revisión de requerimientos y de código, análisis estático) y dinámico, los principios del testing y las características de calidad del modelo ISO/IEC 25010 priorizadas para el SISTEMA, utilizando con precisión los términos error, defecto, falla y bug.

Requerimientos, niveles de prueba y casos de prueba: análisis de los requerimientos del SISTEMA, definición de los niveles de prueba y de los criterios de aceptación (UAT), y casos de prueba derivados de casos de uso con su estructura completa (identificador, precondiciones, datos de prueba, pasos, resultado esperado y trazabilidad al requisito).

Pruebas de caja negra y de caja gris : partición de equivalencia, valores límite, tablas de decisión y transición de estados, ejecutadas de forma manual y documentadas con evidencia, y pruebas de caja gris cuando corresponda (por ejemplo, la verificación de los datos persistidos o de las respuestas de la API).

Pruebas de caja blanca: diseño de casos a partir del código fuente aplicando cobertura de sentencias, decisiones y caminos, implementados con JUnit 5 y medidos con una herramienta de cobertura (JaCoCo).

Estrategias de automatización, testing ágil y TDD : criterios para decidir qué pruebas automatizar y cuáles no, suites automatizadas con JUnit 5, y desarrollo de una parte del SISTEMA o de las correcciones de defectos aplicando el ciclo TDD (red-green-refactor), dejando evidencia de cada etapa del ciclo.

Gestión de defectos, entornos y Definition of Done: reporte de cada defecto con severidad, prioridad, evidencia y seguimiento de su ciclo de vida en Jira, descripción de los entornos de prueba y definición del Definition of Done del equipo.

Plan de pruebas, pairwise y KPIs de QA: aplicación de la técnica pairwise para reducir combinaciones de parámetros, control de cambios, ciclo de release y medición de los indicadores de QA definidos en el plan.

BDD y testing de mutación: escenarios en formato Given-When-Then automatizados con Cucumber, y evaluación de la calidad de la suite de pruebas mediante PIT, analizando los mutantes sobrevivientes y las mejoras realizadas.

Automatización de pruebas web y de APIs : colecciones de Postman con variables de entorno y verificaciones automáticas (assertions) sobre los endpoints del SISTEMA y automatización de flujos web con Selenium cuando el SISTEMA cuente con interfaz web.

Pruebas y artefactos adicionales: el equipo podrá incorporar las pruebas, los artefactos y las herramientas adicionales que considere pertinentes para su estrategia, siempre que hayan sido vistos en clase o indicados por los docentes durante la cursada (por ejemplo, Playwright u otras herramientas de testing que los docentes definan), y que estén justificados en el plan de pruebas.

Métricas y análisis de resultados: El equipo deberá medir y analizar, como mínimo, la cobertura de código, los resultados de la ejecución de los casos (aprobados, fallidos y bloqueados), los defectos detectados por severidad y el resultado del testing de mutación, comparándolos con los criterios de salida definidos en el plan de pruebas y concluyendo sobre el estado de calidad del SISTEMA.

Informe final: El equipo deberá elaborar un informe que resuma la estrategia aplicada, los resultados obtenidos, los defectos hallados, los riesgos residuales, las limitaciones del trabajo y las recomendaciones sobre la liberación del producto.

Comentarios y documentación: El código de pruebas debe estar comentado adecuadamente, explicando el propósito de cada prueba y de los módulos propuestos. Los artefactos de prueba deberán ser claros, estar identificados y mantener la trazabilidad entre requisitos, casos de prueba y defectos.

4. Especificaciones Técnicas

- **Lenguaje de programación:** Java, utilizando una versión compatible con la configuración de Spring Boot definida para el proyecto (Java 21).
- **Framework y entorno de desarrollo:** Spring Boot como framework principal del SISTEMA y de las pruebas de integración. Para la implementación y exposición del Trabajo Práctico Integrador en laboratorio se podrá utilizar Visual Studio Code (VS Code).
- **Pruebas unitarias, de integración y de API automatizadas:** JUnit 5 junto con Spring Boot Test y MockMvc para automatizar pruebas de componentes, servicios, controladores y endpoints del SISTEMA.
- **TDD y BDD:** ciclo red-green-refactor para el desarrollo guiado por pruebas, y Cucumber para especificar y automatizar escenarios Given-When-Then.
- **Testing de mutación:** PIT para evaluar la capacidad de la suite de pruebas de detectar defectos.
- **Pruebas de APIs y web:** Postman para las pruebas manuales y automatizadas de los endpoints de las APIs REST, y Selenium para la automatización de pruebas web.
- **Gestión de defectos y tareas:** registro, seguimiento y el ciclo de vida de los defectos y de las tareas de testing.
- **Técnica combinatoria:** técnica pairwise, con la herramienta generadora de combinaciones que elija el equipo.
- **Herramientas adicionales:** los docentes podrán indicar durante la cursada herramientas, técnicas o artefactos adicionales de testing, que pasarán a formar parte de estas especificaciones técnicas y podrán ser incorporados a la estrategia de pruebas del equipo.

- **Exposición en laboratorio**

  La presentación y demostración del Trabajo Práctico Integrador deberá realizarse sobre el entorno de desarrollo utilizado por el estudiante, mostrando la ejecución del SISTEMA, de las pruebas manuales y automatizadas y de los reportes generados (cobertura, mutación, escenarios BDD y colecciones de Postman), la gestión de los defectos y, cuando corresponda, la interacción con la base de datos y los servicios desarrollados.

5. Entrega

El proyecto debe ser exportado y entregado en formato .zip, y debe incluir el código fuente del SISTEMA y todos los artefactos de verificación y validación:

- Código fuente de las pruebas automatizadas (unitarias, de integración, de API, BDD y web, según la estrategia definida).
- Plan de pruebas y estrategia de testing.
- Casos de prueba y matriz de trazabilidad entre requisitos, casos de prueba y defectos.
- Evidencia de ejecución de las pruebas manuales (resultados y capturas).
- Reportes de defectos.
- Reportes generados por las herramientas (cobertura de JaCoCo, mutación de PIT, escenarios de Cucumber y colección y entorno de Postman exportados).
- Informe final con las métricas, el análisis de resultados y las conclusiones.

Además, el proyecto deberá compilar y funcionar de manera correcta, y las pruebas automatizadas deberán ejecutarse correctamente mediante Maven: un resultado fallido solo se admite cuando evidencia un defecto del SISTEMA que se encuentre debidamente reportado. No se aceptarán los proyectos incompletos o que presenten fallos de funcionamiento de las propias pruebas, de lógica de programación, que se encuentren desarrollados en otras tecnologías o que tengan comportamientos inesperados. No se aceptarán proyectos entregados fuera de la fecha de entrega establecida.

6. Consignas

Una vez asignada la propuesta, se deberá planificar, diseñar, ejecutar, automatizar, medir y reportar la verificación y validación del SISTEMA, definiendo la estrategia de pruebas y seleccionando los artefactos y las pruebas que el equipo considere pertinentes entre los contenidos vistos en la asignatura, y justificando cada decisión.

Además, se deberá definir y documentar, para cada escenario de prueba, la información correspondiente (precondiciones, datos de prueba, pasos, resultado esperado y criterio de aprobación), con el propósito de que las pruebas sean claras, repetibles y trazables.

Los docentes podrán brindar durante la cursada indicaciones, herramientas o criterios adicionales que los equipos deberán considerar en su estrategia de pruebas.

7. Evaluación

El equipo o integrantes que no se presenten el día de la exposición del trabajo práctico integrador quedarán de manera automática descalificados y no podrán aprobar o promocionar la asignatura ya que la misma es una instancia evaluatoria obligatoria y necesaria para la aprobación de la materia. Se aplica la misma norma descrita para el día de la evaluación individual.

El proyecto se evaluará en función de los siguientes criterios:

1. **Estrategia y plan de pruebas (20%):** El plan y la estrategia son coherentes con el SISTEMA, están justificados y definen alcance, riesgos, criterios de entrada y de salida y entornos.
2. **Diseño de casos y aplicación de técnicas (20%):** Los casos de prueba están correctamente diseñados con las técnicas de caja negra, caja gris y caja blanca vistas, son trazables a los requisitos y cuentan con evidencia de ejecución.
3. **Automatización y calidad del código de prueba (20%):** Las pruebas automatizadas funcionan, son repetibles, están bien estructuradas y comentadas y siguen buenas prácticas (TDD, BDD, pruebas de API y web, según la estrategia).
4. **Gestión de defectos, métricas e informe final (10%):** Los defectos están bien reportados y gestionados, y las métricas (cobertura, mutación y KPIs) se analizan con criterio y sustentan las conclusiones.
5. **Creatividad y pruebas adicionales (10%):** Se valorará la incorporación de pruebas, artefactos o herramientas adicionales no requeridas, pertinentes y justificadas, incluyendo el uso crítico de la Inteligencia Artificial.
6. **Exposición oral del equipo de trabajo (20%):** La duración de la misma deberá desarrollarse en un periodo de tiempo máximo de 15 minutos y la distribución del tiempo deberá ser de manera equitativa dependiendo de la cantidad de los integrantes, de manera que todos deben exponer el proyecto integrador.

8. Fecha de revisiones

La fecha para la inspección y corrección y/o modificaciones del trabajo práctico es el [**30-10-2026**].

9. Fecha de entrega

La fecha límite para la entrega del trabajo práctico es el **[13-11-2026**].

10. Lineamientos

Deberán entregar y ajustar el nombre del proyecto suministrado en formato ZIP, respetando las siguientes indicaciones:

- **VV-<cuatrimestre>-<año>-<turno>-<equipo>-<nombre de proyecto>**

- Se deberán reemplazar las secciones resaltadas con los datos apropiados.
- Ejemplo: VV-Q2-2026-TN-Softech-SIGVeterinaria

Se deberá entregar la instancia evaluatoria práctica grupal en el campus virtual de la cátedra (en la práctica determinada para tal fin) una vez finalizado y/o solicitado por el docente. Si no se ha hecho la entrega del mismo por descuido o se realiza la entrega de otro tipo de documento, el mismo será anulado sin excepción.

Verificar que el proyecto esté correctamente enviado y entregado en la sección de trabajos prácticos integradores, ya que, en base a esa entrega se realizarán las observaciones y correcciones de las mismas.

La versión de Java del proyecto deberá ser ajustada a la solicitada por la cátedra (Java 21).

11. Condiciones de aprobación

Se considerará desaprobado todo proyecto que:

a. no compile, o las pruebas automatizadas no se ejecuten correctamente, o,
b. no se identifiquen el SISTEMA, su alcance y los requisitos necesarios para resolver lo solicitado, o,
c. las técnicas de diseño, los datos de prueba o las herramientas elegidas no sean los adecuados para lo que se prueba, o no cumpla con las condiciones mínimas solicitadas por la especificación de requisitos en el enunciado (plan de pruebas, casos de prueba, reportes de defectos o informe final).
d. La entrega esté vacía o se envíe otro documento por error, sin excepción.
e. El equipo de desarrollo no se haya registrado en la planilla de equipos de proyectos, no tengan un nombre asignado y/o no tengan elegido un proyecto a desarrollar.
f. Se permite el uso de herramientas de inteligencia artificial únicamente como apoyo a las actividades de testing con IA desarrolladas en la asignatura y de acuerdo con las indicaciones de los docentes. Su uso deberá declararse y documentarse (herramienta utilizada, instrucciones brindadas, resultado obtenido y verificación realizada por el equipo). No se admite entregar resultados generados por IA sin revisión ni validación del equipo, ni delegar en ella la resolución completa del trabajo: la resolución deberá sustentarse en los conocimientos y contenidos desarrollados durante la cursada y la bibliografía indicada por la cátedra. El incumplimiento invalida la entrega.

Ing Di Ludovico Maximiliano,
Ing Millan Alejandro
— Verificación y Validación de Software
Escuela de Ciencia y Tecnología (ECyT)
Universidad Nacional de San Martín (UNSAM)